$Logical\ (Programmed) Pop\ vs\ Hardware\ Instruction\ POP$

$In\ Logical\ Programmed\ Pop\ operation$:

```
int pop(Stack *st)
{
    if (isEmpty(*st))
    {
        cout << "Stack Underflow" << endl;
        return -1;
    }
    return st->s[st->top--];
}
```

$What\ it\ does$?

$First$: $st \rightarrow s[st \rightarrow top]$ $and\ then$: $Top = Top - 1$,

$That\ is\ we\ first\ print\ or\ output$: $st \rightarrow s[st \rightarrow top]$, $and\ an\ ALU$

$Operation$: $Top = Top - 1$, $and\ it\ is\ executed\ later\ due\ to$

$Post\ Decrement$. $What\ happens\ here\ is$:

==========

THE C++ ARRAY STACK

==========

1. *Compiler reads*: *return stack*[*top* − −];

2. *It generates these standard machine instructions* (*Not a native POP*!):

[*Address*] [*Instruction*]

[0x1000] −> *LOAD the integer value of 'top' from memory into Reg*1

[0x1004] −> *CALCULATE the memory address of stack*[*top*]

[0x1008] −> *READ the data* (*e.g.*, 30) *from that array address into Reg*2

[0x100C] −> *SUBTRACT* 1 *from Reg*1 (*top* = *top* − 1)

[0x1010] −> *WRITE the new 'top' value back to memory*

==========

TEXT / CODE SEGMENT (*Where the Program Counter points*)

==========

(*The PC walks down these addresses one by one*)

[0x1000] −> [*Instruction*: *LOAD 'top' into Reg*1]

[0x1004] −> [*Instruction*: *CALCULATE address of stack*[*top*]]

[0x1008] −> [*Instruction*: *READ data from array into Reg*2]

[0x100C] −> [*Instruction*: *SUBTRACT* 1 *from Reg*1]

[0x1010] −> [*Instruction*: *WRITE new 'top' back to memory*]

Now how this is seen in actual Memory:

==========

$\boldsymbol{TEXT\ /\ CODE\ SEGMENT\ (Instruction\ Memory)}$

==========

$[\ \boldsymbol{Address}\ ]\quad [\ \boldsymbol{16-bit\ Machine\ Code}\ ]\quad [\ \boldsymbol{CU\ Translation\ (Invisible)}\ ]$

$[\ \boldsymbol{0x1000}\ ] \text{->}\ [\ \boldsymbol{0100\ 100000000000}\ ]\ <--\ \boldsymbol{Opcode}: \boldsymbol{0100}\ (\boldsymbol{LOAD}), \boldsymbol{Operand}: \boldsymbol{0x800}$

$[\ \boldsymbol{0x1004}\ ] \text{->}\ [\ \boldsymbol{0111\ 000001010000}\ ]\ <--\ \boldsymbol{Opcode}: \boldsymbol{0111}\ (\boldsymbol{CALC}), \boldsymbol{Operand}: \boldsymbol{Base\ Array\ Addr}: \boldsymbol{0x050}$

$[\ \boldsymbol{0x1008}\ ] \text{->}\ [\ \boldsymbol{0100\ 000000000000}\ ]\ <--\ \boldsymbol{Opcode}: \boldsymbol{0100}\ (\boldsymbol{LOAD}), \boldsymbol{Operand}: (\boldsymbol{Calculated\ Addr})$

$[\ \boldsymbol{0x100C}\ ] \text{->}\ [\ \boldsymbol{0101\ 000000000001}\ ]\ <--\ \boldsymbol{Opcode}: \boldsymbol{0101}\ (\boldsymbol{SUB}),\qquad \boldsymbol{Operand}: \boldsymbol{1}$

$[\ \boldsymbol{0x1010}\ ] \text{->}\ [\ \boldsymbol{0110\ 100000000000}\ ]\ <--\ \boldsymbol{Opcode}: \boldsymbol{0110}\ (\boldsymbol{STORE}), \boldsymbol{Operand}: \boldsymbol{0x800}$

$\boldsymbol{1st\ 0100(Load)\ then}: \boldsymbol{1000_2 = 0x8_{16}, 0000_2 = 0x0_{16}, 0000_2 = 0x0_{16} \Rightarrow 0x800}, \boldsymbol{its\ the\ address\ where\ value\ of\ variable\ TOP\ is\ stored}, \boldsymbol{say\ TOP = 4\ stored\ at\ 0x800\ and\ hence\ 4\ loaded\ from\ the\ correspondence\ address}: \boldsymbol{0x800}.$

$\boldsymbol{2nd\ 0111(Calc)\ then}: \boldsymbol{0000_2 = 0x0_{16}, 0101_2 = 0x5_{16}, 0000_2 = 0x0_{16} \Rightarrow 0x050}, \boldsymbol{Calculate\ address\ of\ stack[top]}, \boldsymbol{i.e.\ from\ Base\ Address}: \boldsymbol{0x050_{16}}.$

$\boldsymbol{And\ this\ happens\ in\ ALU\ (Arithmetic\ Logic\ UNIT)}:$

==========

THE ARRAY CALCULATION (Inside the ALU)

==========

[***Operand***: **0x050**] ----------> [***Base Address***: **0x050**]

↓

[***Reg1 (holds index '4')***] ---------> [***Index***: **4**] ∗ [***Size of int***: **4**]

↓

[***ALGORITHM AND LOGIC UNIT***] <------ ***Performs***: **0x050 + (4 ∗ 4)**

↓ **(0x050 + 16 *in decimal*)**

(0x050 + 0x10 *in hex*)

[***Final Array Address***: **0x060**]

3rd 0100(LOAD) then: 0000_2 , 0000_2, 0000_2 – ***When this reaches to Instruction Register(IR) Control Unit splits the data reads OPCODE*** , **0100** ***as LOAD and recognizes as INDIRECT LOAD and ignores the 12 bit operand*** : **0000 0000 0000** ***completely***.

The compiler cannot hardcode the address 0x060 into the instruction because 0x060 is just a temporary result. If the top index changes to 3 tomorrow, the address will be different. The compiler puts 000000000000 there purely as a dummy placeholder.

$0000_2 = 0x0, 0000_2 = 0x0 , 0000_2 = 0x0 \Longrightarrow 0x000$ – ***this is a dummy placeholder***.
Because the CU knows to ignore the 0x000 placeholder, it turns its attention to the calculated address (0x060) that we just pushed into the Memory Address Register (MAR) in the last cycle.

The setup is now perfect: The MAR securely holds the correct map (0x060).

This occurs as Dynamic Runtime Calculation(often termed as ``Indirect Addressing``)
As dummy address is placed purposely to be ignored in the dynamic runtime mode.

==========

THE PRECISE INTERNAL ROUTING (ALU –> MAR)

==========

[***CONTROL UNIT (CU)***]

|

| – – – (***Signal 1***: "***Output Enable***")

| – – – – – – –> ***Sent to*** [***ALU Output Buffer***] : "***Release the 0x060***!"

|

| – – – (***Signal 2***: "***Load Enable***")

| – – – – – – –> ***Sent to*** [***MAR***] : "***Catch the 0x060***!"

– –

THE PHYSICAL PATHWAY (Internal Logic Gates)

– –

[***ALU Output***] ***Holds***: ***0x060***

↓ (***Flows across internal CPU wiring***)

[***Memory Addr. Reg (MAR)***]

(***Load Enable gate opens, latches the address***)

Now Holds: ***0x060***

==========

$THE\ DATA\ TRANSFER\ THROUGH\ ADDRESS$

==========

$-- \ STEP\ 1: Using\ the\ Address\ --$

$[\ ALU\ ]\ Calculates\ Address: 0x060$

$\downarrow (Internal\ Routing)$

$[\ MAR\ ]\ Now\ Holds: 0x060\ \ <--\ (The\ Map)$

$-- \ STEP\ 2: Fetching\ the\ Data\ --$

$[\ RAM\ Slot\ , STACK\ ADDRESS: 0x060\ ]\ Holds\ Data: 30$

$\downarrow (Via\ Data\ Bus\ and\ through\ \ MDR)$

$[\ Reg\ 2\ ]\ Now\ Holds: 30\ \ <--\ (The\ Treasure!)$

$\underline{Now\ , REG\ 1\ of\ CPU\ hold\ , TOP\ Value - 4(Say)}$

$As, 0101\ 000000000001\ reaches\ IR(Instruction\ Register) of\ CPU, CU\ reads$

$0101\ as\ SUB\ and\ 12\ bit\ OPERAND:\ 000000000001\ as\ 1\ , that\ subtract\ `1`\ from$

$4\ [000000000100]\ which\ is\ in\ REG\ 1\ of\ CPU\ .$

Step 1: The Decode & Route Setup

When the 16-bit instruction is sitting in the IR, the CU acts as the ultimate traffic controller. It decodes the `0101` and realizes, *"This is an immediate math operation. I don't need to talk to RAM. I need the ALU."*

- **Route Operand:** The CU extracts the 12-bit operand (`000000000001`) straight from the IR and routes it directly to **Input Bus B** of the Algorithm and Logic Unit (ALU).
- **Route REG 1:** The CU fires an **"Output Enable"** signal to REG 1. This opens the gates of REG 1, allowing its stored binary `000000000100` to flow down the internal wires to **Input Bus A** of the ALU.

Step 2: The ALU Execution

Both numbers are now physically sitting at the entrance gates of the ALU.

- The CU fires a specific control signal down a wire directly into the ALU telling it to configure its logic gates for **SUBTRACTION**.
- The ALU processes the binary math at the speed of electricity: `000000000100` - `000000000001`.
- The ALU instantly outputs the binary result: `000000000011` (which is decimal 3).

Step 3: The Latch (Saving the Result)

The ALU does not have permanent memory; it just holds the `000000000011` on its output wires. If the CPU moves on, that data will vanish.

- The CU routes this output back to REG 1.
- The CU fires a **"Load Enable"** (or Latch) signal directly to REG 1.
- REG 1 opens its input gates, catches the `000000000011`, and physically overwrites the old `4` with the new `3`.

==========

THE INTERNAL SUBTRACTION EXECUTION $(top--)$

==========

[***Instruction Register*** (IR)]

$|--$ ***Opcode***: **0101** (SUB) $----->$ $($***Tells CU to set ALU to Subtract***$)$

$|--$ ***Operand***: **000000000001** $-->$ $($***Routed directly to ALU Input B***$)$

[***General Purpose REG*** **1**] ***Holds***: **000000000100** $($***Decimal*** **4**$)$

$\downarrow$ $($***CU fires "Output Enable" to REG*** **1**$)$

[***ALU Input Bus A***: **0100**] [***ALU Input Bus B***: **0001**]

$\searrow$ $\swarrow$

[***ALGORITHM AND LOGIC UNIT*** (ALU)]

$($***CU fires "SUBTRACT" signal***$)$

Performs: **0100 − 0001**

$\downarrow$

[***Output***: **0011**] $($***Decimal*** **3**$)$

$\downarrow$ $($***CU fires "Load Enable" to REG*** **1**$)$

[***General Purpose REG*** **1**] ***Overwritten***! ***Now Holds***: **000000000011** (3)

The Current State of the Machine

The `0101` instruction is now 100% complete! The CPU is holding the two critical pieces of data from your C++ code:

1. **REG 2** holds the extracted array data: `30` (from the previous steps).
2. **REG 1** holds the newly decremented index: `3`

$\boldsymbol{Now\ as\ ,0110\ 100000000000\ reaches\ to\ IR\ ,it\ is\ read\ as\ 0110\ as\ STORE\ and\ 100000000000}$

$\boldsymbol{,1000_2 = 0x8_{16}, 0000_2 = 0x0_{16}\ ,0000_2 = 0x0_{16} \Longrightarrow 0x800\ ,i.e.STORE\ the\ 3\ in\ REG\ 1\ to\ 0x800,}$

$\boldsymbol{the\ stack\ address\ where\ TOP's\ value\ 4\ is\ stored.}$

Step 1: Routing the Address to the Outbound Door (MAR)

The Instruction Register (IR) currently holds `0110 100000000000`. The CU reads `0110` and knows this is a `STORE` command. It immediately looks at the 12-bit operand to find out *where* to store the data.

The CU takes that address (`0x800`) and sends it directly to the Memory Address Register (MAR).

$$==========$$

$$\boldsymbol{PHASE\ 1: ADDRESS\ ROUTING\ (Inside\ the\ CPU)}$$

$$==========$$

$$[\ \boldsymbol{Instruction\ Register\ (IR)}\ ]$$

$$|--\boldsymbol{Opcode}: \boldsymbol{0110}\ (\boldsymbol{STORE})$$

$$|--\boldsymbol{Operand}: \boldsymbol{100000000000}\ (\boldsymbol{0x800})$$

$$\Big\downarrow\ (\boldsymbol{CU\ routes\ Address\ via\ internal\ wires})$$

$$[\ \boldsymbol{Memory\ Addr.Reg\ (MAR)}\ ]\ \boldsymbol{Now\ safely\ holds}: \boldsymbol{0x800}$$

Step 2: Routing the Data to the Outbound Door (MDR)

Now the CPU knows the destination, but it needs to move the actual payload (`3`) to the exit. Because all data leaving the CPU must go through the Memory Data Register (MDR), the Control Unit opens an internal pathway.

It routes the operand (the result `3`) from REG 1 straight into the MDR. It does this by firing an "Output Enable" signal to REG 1 and a "Load Enable" signal to the MDR.

==========

$\boldsymbol{PHASE\ 2: DATA\ ROUTING\ (Inside\ the\ CPU)}$

==========

[$\boldsymbol{General\ Purpose\ REG\ 1}$] $\boldsymbol{Holds}$: $\mathbf{000000000011}$ ($\boldsymbol{Decimal}$ $\mathbf{3}$)

$\downarrow$ ($\boldsymbol{CU\ routes\ Data\ via\ internal\ wires}$)

[$\boldsymbol{Memory\ Data\ Reg\ (MDR)}$] $\boldsymbol{Now\ safely\ holds}$: $\mathbf{3}$

Step 3: The External WRITE Execution (The Final Push)

Everything is perfectly staged at the CPU's exit doors. The MAR holds the destination map, and the MDR holds the physical data payload. Now, the CU pulls the trigger to blast this data out to the RAM.

1. **Address Bus:** The MAR places the address `0x800` onto the Address Bus.
2. **Data Bus:** The MDR pushes the binary data (`3`) out onto the Data Bus.
3. **Control Bus:** The Control Unit sends the `WRITE` signal to the Control Bus, turning it on.

$$==========$$

$$\boldsymbol{PHASE\ 3: THE\ WRITE\ EXECUTION\ (Reaching\ Outside)}$$

$$==========$$

$$\boldsymbol{CPU} \qquad\qquad\qquad\qquad \boldsymbol{MAIN\ MEMORY}$$

$$[\ \boldsymbol{MAR}\ ]\ ---\ (\boldsymbol{Address\ Bus}: \mathbf{0}\boldsymbol{x}\mathbf{800})\ --->\ [\ \boldsymbol{Address\ Decoder}\ ]$$

$$|$$

$$[\ \boldsymbol{CU}\ ]\ ---\ (\boldsymbol{Control\ Bus}: \boldsymbol{WRITE}\ )\ --->\ [\ \boldsymbol{Open\ the\ gates}!\ ]$$

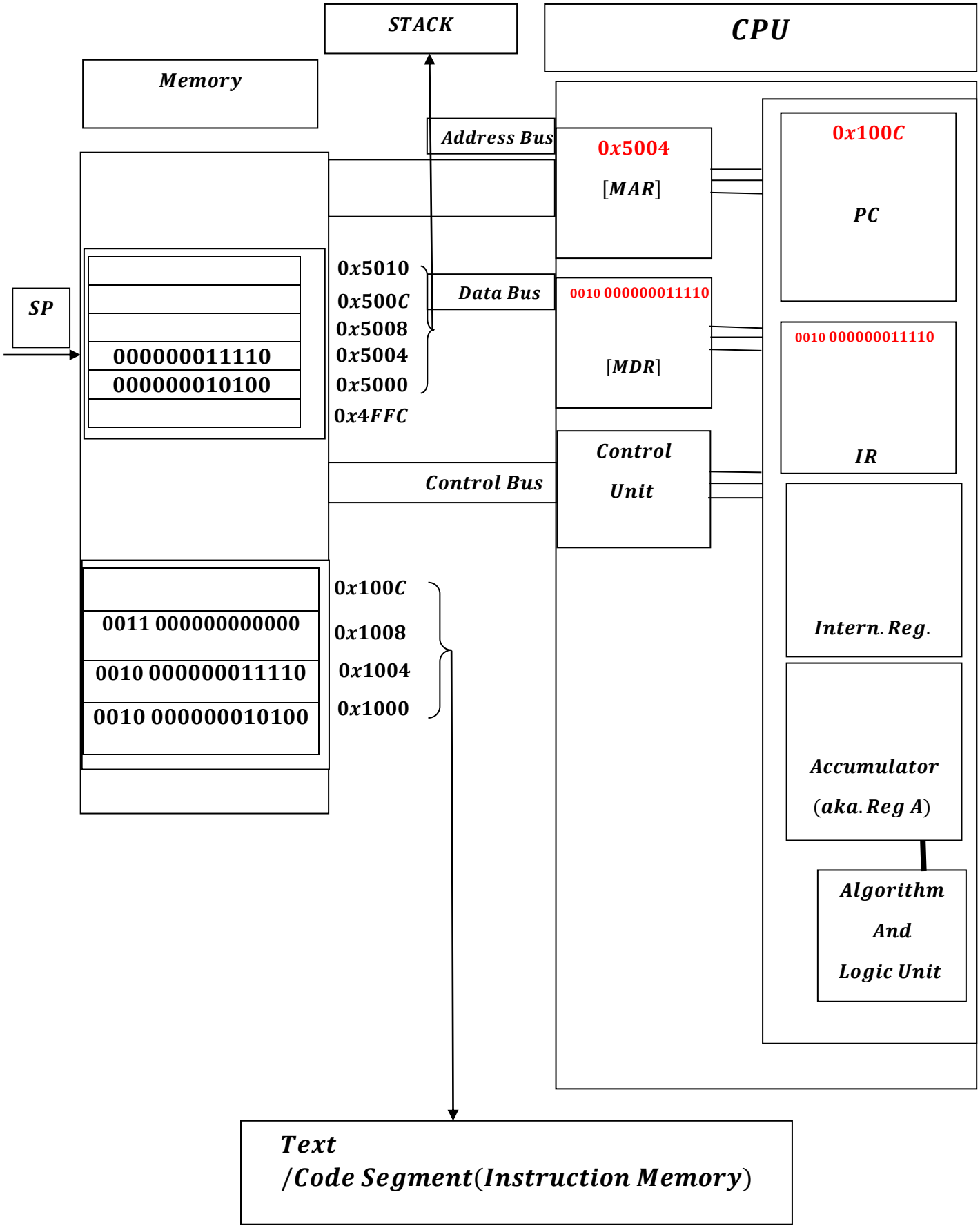
$$|$$

$$[\ \boldsymbol{MDR}\ ]\ ---\ (\boldsymbol{Data\ Bus}: \mathbf{3})\ --->\ [\ \boldsymbol{Overwrite}\ \mathbf{0}\boldsymbol{x}\mathbf{800}\ ]$$

When the Main Memory receives that high-voltage WRITE signal, it navigates to physical slot 0x800. It catches the 3 from the Data Bus and locks it into the transistors. The old data (4) that was previously sitting at 0x800 is utterly destroyed and overwritten by the new sum.

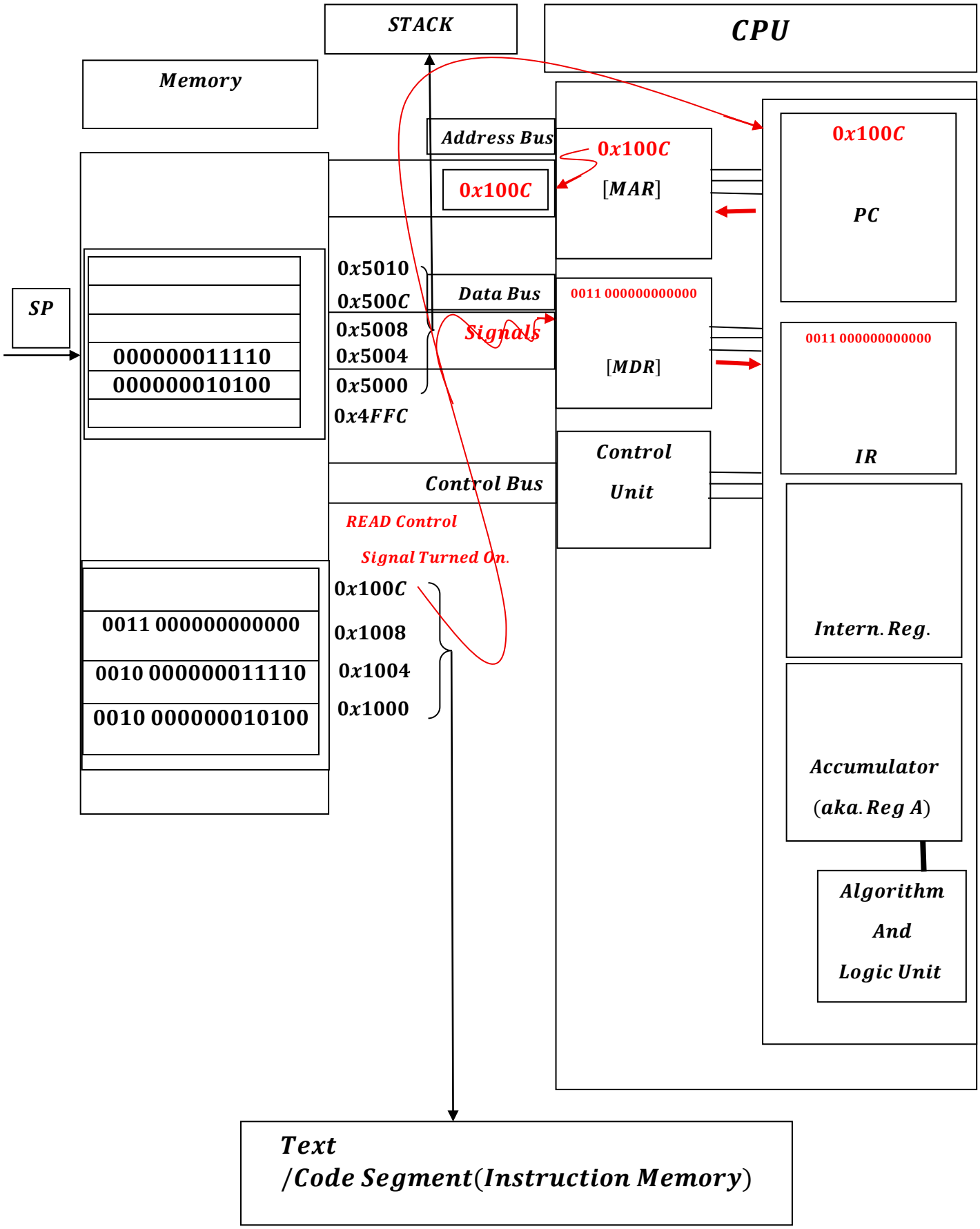
The REG 1 and REG 2 can be renamed as REG A and REG B respectively . And one can *also make REG A as ACCUMULATOR to store the data* : 3.

THE HARDWARE instruction native POP



Now on, $0x1008$ *of TEXT /CODE SEGMENT we have* 0011 *(4 bit OPCODE of POP)*

$12 -$ *bit zeroes* 000000000000_2 *which are to be ignored by CU.*



==========

$THE\ 1 - STEP\ HARDWARE\ NATIVE\ POP\ (Opcode: 0011)$

==========

[$Instruction\ Register\ (IR)$] $Holds: 0011\ 000000000000$

$\downarrow$

[$CONTROL\ UNIT$] ($Reads\ 0011\ and\ activates\ the\ hardwired\ "POP\ Recipe"$)

|

|--- ($Action\ 1: Address\ Routing\ -\ NO\ ALU\ NEEDED!$)

|-------> $Routes\ the\ SP\ Address\ (e.g., 0x5004)\ directly\ to\ MAR.$

|

|--- ($Action\ 2: Fetch\ Data\ -\ 1\ CYCLE!$)

|-------> $Turns\ on\ READ\ signal\ on\ Control\ Bus.$

|-------> $Memory\ sends\ Data\ (30)\ to\ MDR.$

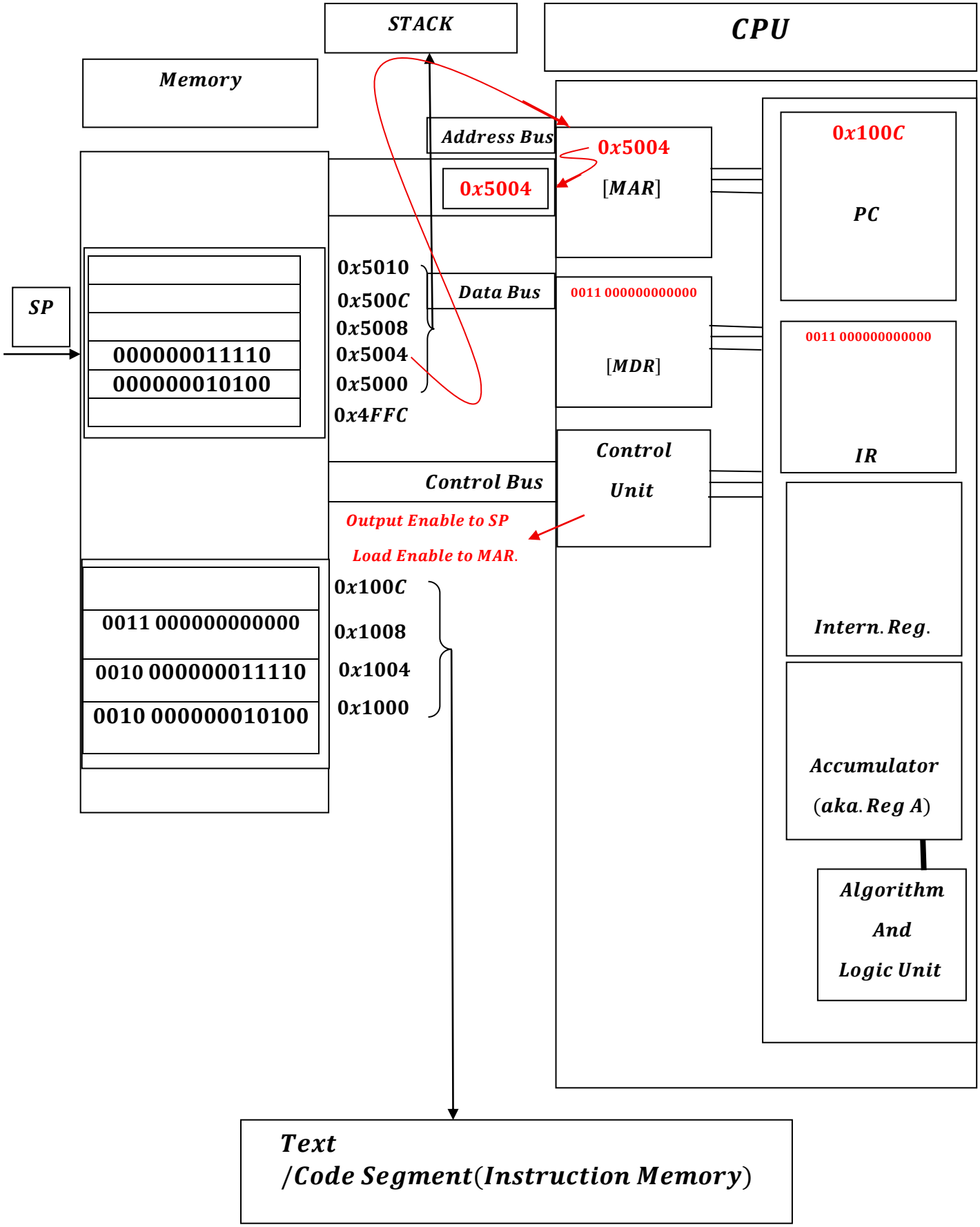
|-------> $CU\ routes\ Data\ (30)\ from\ MDR\ into\ a\ General\ Purpose\ Reg.$

|

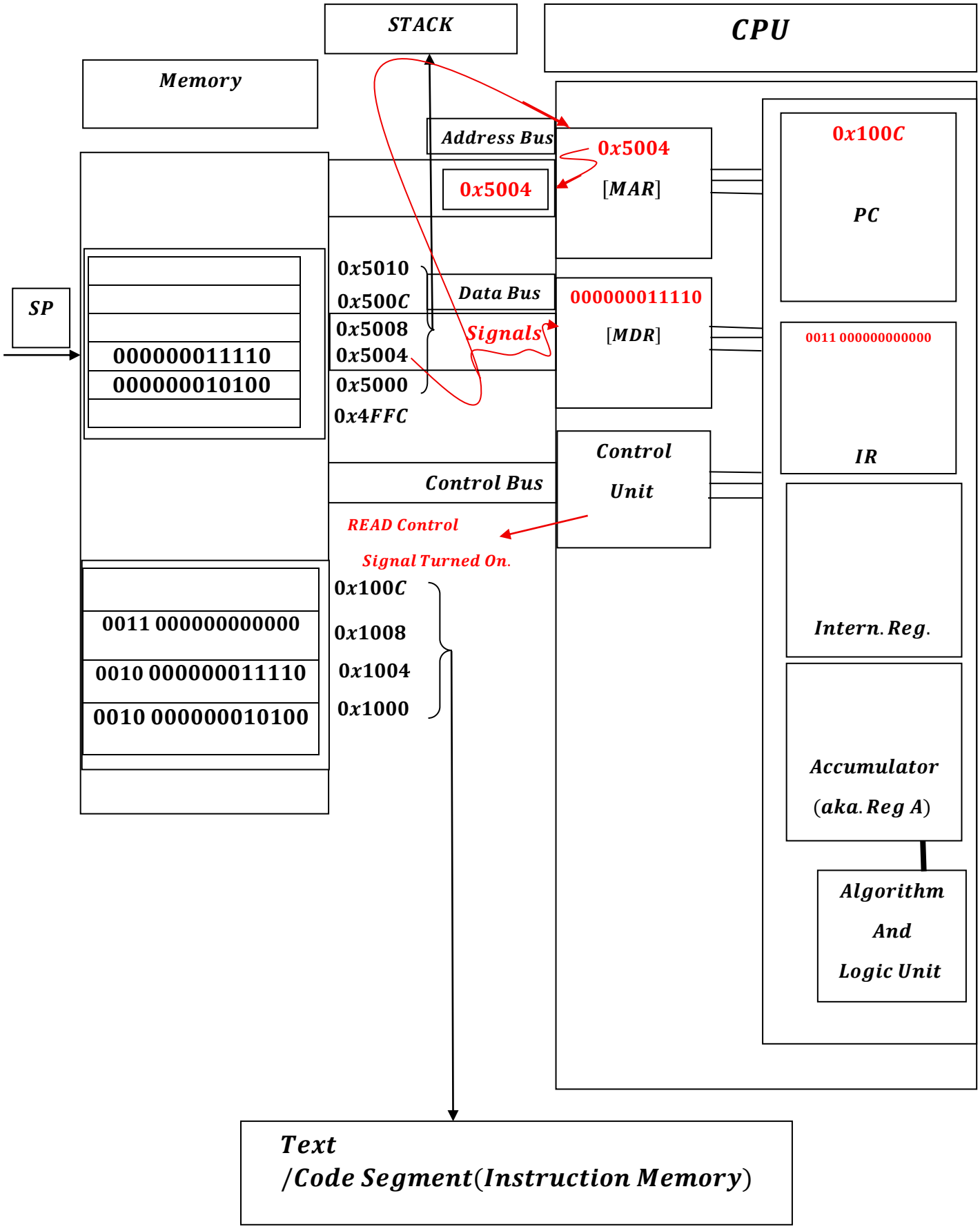
|--- ($Action\ 3: Decrement\ -\ NO\ ALU\ NEEDED!$)

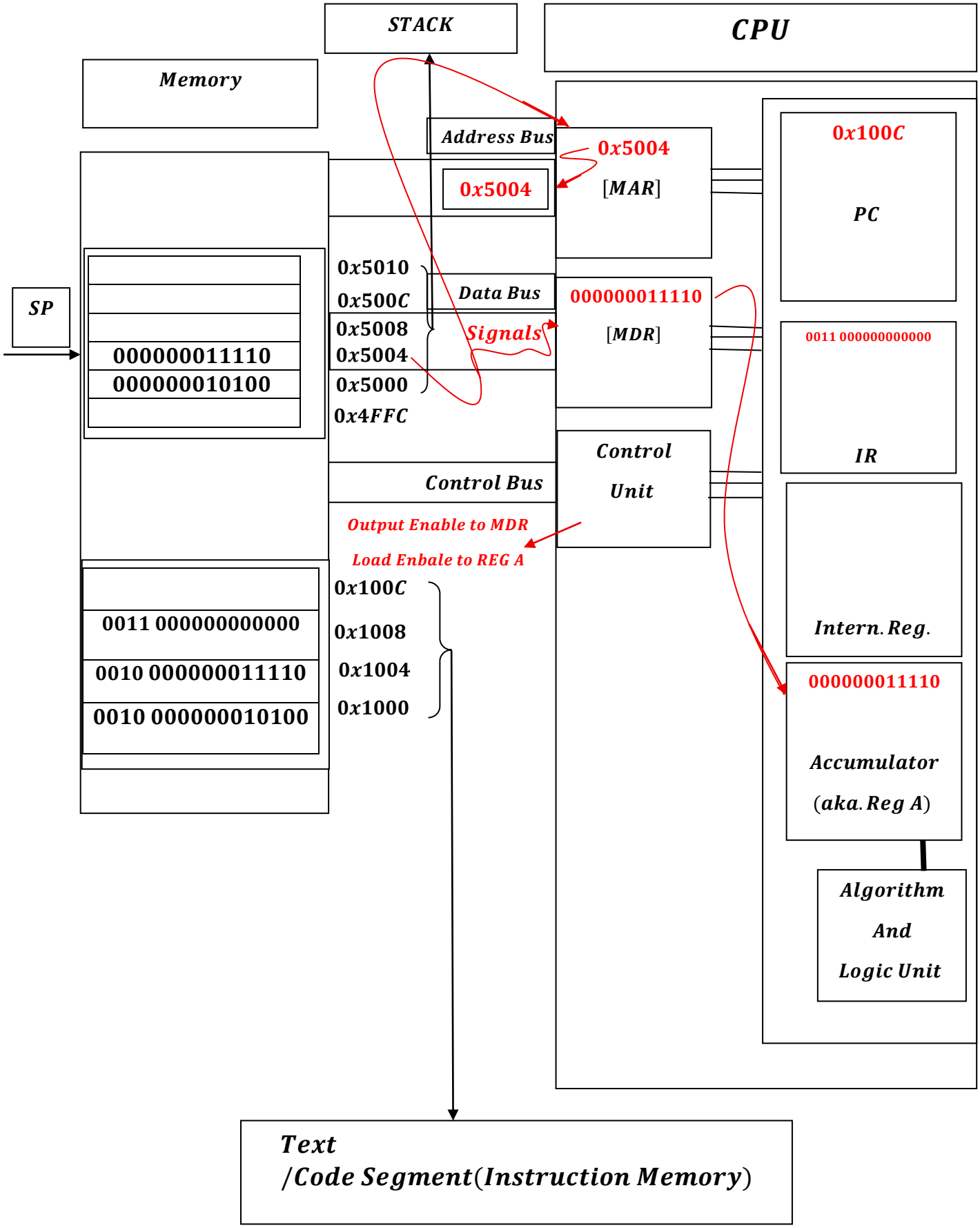
|-------> $Sends\ electrical\ pulse\ to\ SP's\ dedicated\ hardware\ subtractor.$

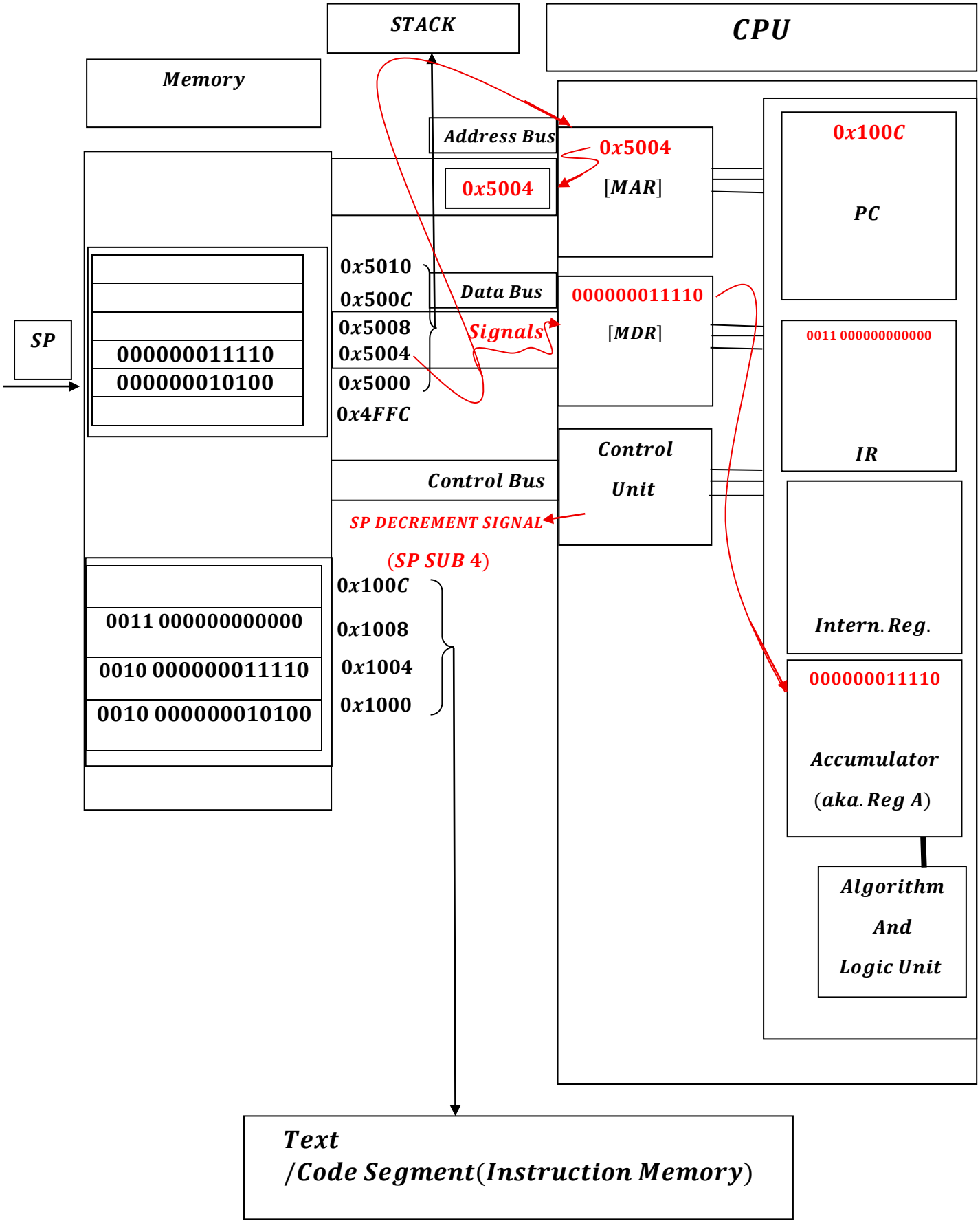
|-------> $SP\ automatically\ calculates: 0x5004\ -\ 4\ =\ 0x5000.$


STACK
CPU
Memory
Address Bus
0x5004
[MAR]
0x5004
0x100C
PC
SP
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
0011 000000000000
[MDR]
000000011110
000000010100
0011 000000000000
IR
Control Unit
Control Bus
Output Enable to SP
Load Enable to MAR.
0x100C
0x1008
0x1004
0x1000
0011 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

CU signals Output Enable to SP register to release stored address i.e. 0x5004 , then simultaneously CU signals Load Enable/Latch to MAR to store address: 0x5004.




STACK
CPU
Memory
Address Bus
0x5004
0x5004
[MAR]
0x100C
PC
SP
000000011110
000000010100
0x5010
0x500C
0x5008
0x5004
0x5000
0x4FFC
Data Bus
Signals
000000011110
[MDR]
0011 000000000000
IR
Control Unit
Control Bus
Output Enable to MDR
Load Enbale to REG A
0011 000000000000
0010 000000011110
0010 000000010100
0x100C
0x1008
0x1004
0x1000
Intern. Reg.
000000011110
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x5004
[MAR]
0x5004
0x100C
PC
0x5010
0x500C
Data Bus
000000011110
[MDR]
0x5008
Signals
0x5004
0x5000
0x4FFC
SP
000000011110
000000010100
0011 000000000000
IR
Control Unit
Control Bus
SP DECREMENT SIGNAL
(SP SUB 4)
0x100C
0x1008
0x1004
0x1000
0011 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
000000011110
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

As, we see CU sends Stack Pointer decrement signal so that the address stored in Stack Pointer gets decremented to 4 bytes through the help of a subtractor attached to the Stack Pointer. It may vary with the architecture .

Though the Data corresponding to the address stored in Stack Pointer Register send to REG A through MDR , then address stored is decremented by 4 bytes $(sizeOf(int))$ *the data* $30 : 000000011110$ *always stay in the memory , until it get overwritten or stack frame gets destroyed i.e. it doesn't actually POP the data out from the memory which is a big difference between logical programme and hardware native pop instruction.*

Such Hardware Related pop , push , top function doesn't occur when we write code like:

```
int pop(Stack *st)
{
    if (isEmpty(*st))
    {
        cout << "Stack Underflow" << endl;
        return -1;
    }
    return st->s[st->top--];
}
```

It will just execute ALU function.

These hardware native instruction POP$(4$ *bit opcode* $: 0011)$ *, PUSH*$(4$ *bit opcode*: $0010)$ *which we cannot write through program generally , its strictly reserved by compiler and Operating System.*

```
================================
  WHEN THE CPU ACTUALLY USES NATIVE PUSH / POP
====================================

[ Code Segment ]            [ Hardware Execution ]


int main() {
  int x = 5;
  calculateMath();  -> 1. NATIVE PUSH: Saves main()'s address to Stack
                       2. PC jumps to calculateMath()
  int y = 10; <---------
}                        |
                         |
int calculateMath() {    |
  // do math             |
  return; ----------/     3. NATIVE POP: Grabs address from Stack into PC
}
```

1. The Invisible PUSH (Calling the function)

Right before the Program Counter (PC) jumps away to the new function, the Control Unit (CU) executes a native hardware `PUSH`.

- **The Payload:** It doesn't push a variable; it pushes the **Return Address** (the exact memory address in `main()` where the CPU needs to resume later).
- **The Execution:** The hardware Stack Pointer (SP) instantly decrements, and the Return Address is locked into the RAM.

2. The Invisible POP (Hitting `return`)

When your `calculateMath()` function hits the `return;` statement at the very end of its code block, the compiler translates that `return` keyword directly into a native hardware `POP` opcode!

- **The Execution:** The CU fires the exact micro-signals we traced earlier (`0011`).

- **The Destination:** It routes the popped data (the Return Address) out of the RAM, through the MDR, and drops it **directly back into the Program Counter (PC)**.
- **The Result:** Because the PC now holds the old address, the CPU instantly teleports back to `main()` and resumes exactly where it left off!

In the above its said STACK Pointer decrements , why ?

Scenario A: The Ascending Stack

In the architecture we just traced, the stack starts at a low memory address and grows "up" into higher numbers.

- **PUSH:** The CU fires the ADD signal (SP = SP + 4). We are climbing up the memory ladder to put a new box on top.

- **POP:** The CU fires the SUB signal (SP = SP - 4). We are stepping down the ladder after removing a box.

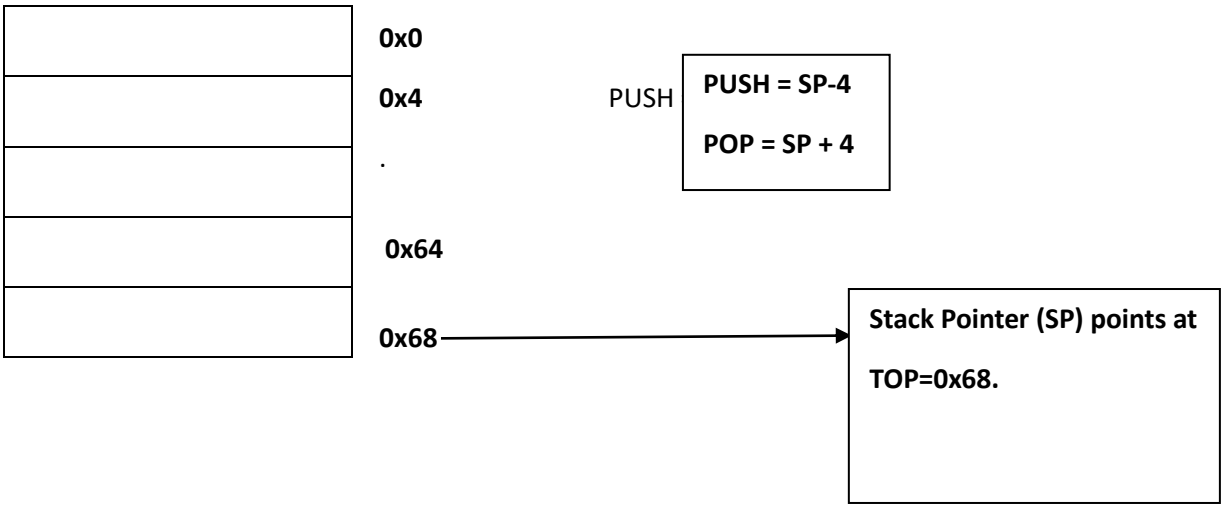
-

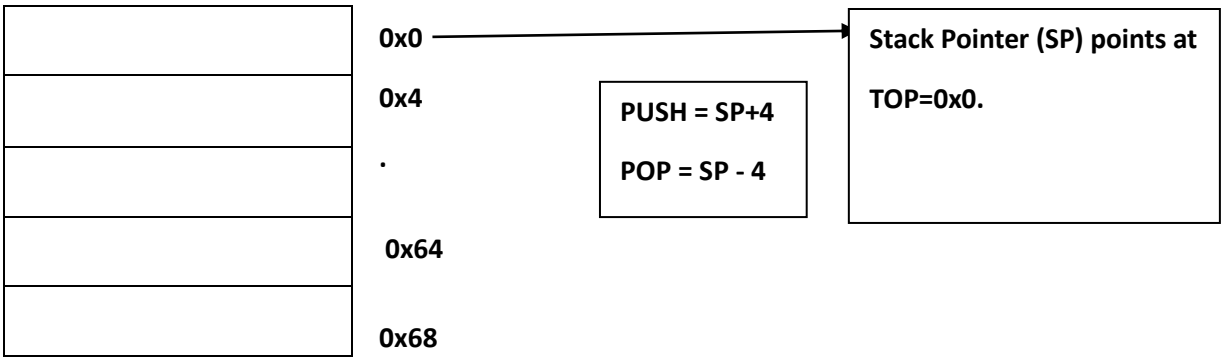
Scenario B: The Descending Stack (Modern CPUs)

This is the wild part about modern engineering! In most everyday, the engineers designed the stack to grow *downwards* from a high memory address towards zero. It feels upside-down, but it prevents the stack from accidentally colliding with our program's code.

- **PUSH:** The CU fires a SUB signal (SP = SP - 4). The pointer drops down to a lower address to claim empty space.

- **POP:** The CU fires an ADD signal (SP = SP + 4). The pointer moves back up into higher numbers as space is freed.





$i.e. Formula\ of\ Stack\ Pointer\ Increment/Decrement: SP \pm (1 \times Size\ of\ (DataType)).$

Here is the exact step-by-step physical route of how the CPU retrieves that Cargo and turns it back into a Map.

Step 1: The MAR Holds the "Map to the Stack"

To get the Return Address off the stack, the CPU first has to know where the top of the stack is.

- The Control Unit (CU) takes the current Stack Pointer address (e.g., 0x5000) and puts it into the **MAR**.
- The MAR broadcasts 0x5000 on the Address Bus. It is telling the RAM: *"Look at slot 0x5000."*

Step 2: The MDR Catches the "Cargo"

The RAM looks inside slot 0x5000. Inside, it finds the saved Return Address (0x1004).

- Because this 0x1004 is being pulled *out* of a storage slot, it must travel on the **Data Bus**.
- The Data Bus only connects to one doorway on the CPU: The **Memory Data Register (MDR)**.
- The MDR catches the 0x1004.

Step 3: The Internal Promotion (MDR → PC)

Now the Return Address 0x1004 is safely inside the CPU's inbound loading dock (the MDR).

To actually use this number as an address and jump back to main(), the CU must promote it. It opens the internal logic gates and routes the 0x1004 directly from the MDR into the **Program Counter (PC)**.

$$==========$$

$$\boldsymbol{HOW\ AN\ "ADDRESS"\ TRAVELS\ AS\ "DATA"\ (The\ POP\ Return)}$$

$$==========$$

$$[\ \boldsymbol{Stack\ Pointer\ (SP)}\ ]\ \boldsymbol{Holds}: \mathbf{0x5000}$$

$$\downarrow$$

$$[\ \boldsymbol{Memory\ Addr.Reg\ (MAR)}\ ]\ ---> \ (\boldsymbol{Address\ Bus}: \mathbf{0x5000})\ ---> \ [\ \boldsymbol{RAM}\ ]$$

$$\downarrow$$

$$(\boldsymbol{RAM\ finds\ stored\ Cargo}: \mathbf{0x1004})$$

$$\downarrow$$

$$[\ \boldsymbol{Memory\ Data\ Reg\ (MDR)}] < --- (\boldsymbol{Data\ Bus\ carries}\ \mathbf{0x1004}) --$$

$$\downarrow\ (\boldsymbol{CU\ routes\ Cargo\ internally})$$

$$[\ \boldsymbol{Program\ Counter\ (PC)}\ ]\ \boldsymbol{Now\ Holds}: \mathbf{0x1004}$$

$$(\boldsymbol{The\ CPU\ instantly\ teleports\ back\ to}\ \mathbf{0x1004}\ \boldsymbol{to\ resume\ the\ program}!)$$

There are inline assembly feature where C++can directly speak to CPU :

1. ***Inline Assembly PUSH***

```
int main()
{
    int myNumber = 42;

    // We are leaving C++ and talking directly to the CPU!
    __asm__(
        "push %0;" // NATIVELY PUSH myNumber onto the
hardware stack!
        :
        : "r"(myNumber));
}
```

When the compiler sees `__asm__`, it stops trying to build software arrays. It directly injects the raw binary `0010` (PUSH) opcode into your executable file!

2. ***Inline Assembly POP***

```
int main()
{
    int poppedValue; // We need an empty variable to catch the treasure!

    // We are leaving C++ and talking directly to the CPU!
    __asm__(
        "pop %0;"           // NATIVELY POP from the hardware stack!
        : "=r"(poppedValue) // OUTPUT section: CPU writes the data here
        :                   // INPUT section: (Empty, we aren't sending
anything)
    );

    // poppedValue now holds whatever was at the top of the hardware stack
}
```

Decoding the Assembly Syntax

When the compiler sees this block, it injects the raw hardware POP opcode into the executable. Here is exactly how the compiler translates our syntax into physical hardware routing:

- **"pop %0;"**: This is the actual instruction. Just like before, %0 is a placeholder. It tells the Control Unit (CU), *"Pop the data off the top of the stack and drop it into the first register ."*
- **The Output Section (:):** In the previous push example, our variable was in the *third* line (the Input section). Here, it is in the *second* line (the Output section). This tells the compiler we are catching data, not sending it.
- **"=r"**: This is the exact micro-routing command!
    - The **r** tells the CU to use a General Purpose **R**egister as the temporary workbench.
    - The **=** tells the CU that this is a **Write-Only** operation. It gives the CPU permission to physically overwrite whatever old garbage data was sitting inside that variable.

What Happens in the Silicon during POP operation?

1. **The Fetch:** The CU routes the Stack Pointer's address to the MAR, fires the READ signal, and the memory pushes the data onto the Data Bus.

2. **The Catch:** The data enters the MDR, and the CU routes it straight into the internal CPU Register that the compiler assigned to poppedValue.

3. **The Update:** The CU sends the electrical pulse to the SP's hardware adder, moving the pointer up (SP = SP + 4) to indicate the box has been removed from the stack. Translating this POP simply describes what is happening in the STACK during the pop operation!